CS433 - Assignment 2

Anuj (210166)

October 2024

1 Files and usage

The following 5 files have been submitted (in accordance with the submission guidelines):

- `sync_library.c`: Contains the implementation of Lock and Barrier functions
- `pthread_main_lock.c`: Written the benchmark code provided. To try different implementations of locks, use the following flags while compilation (default is POSIX mutex):
  - `-DBAKERY_LOCK`: Bakery lock
  - `-DSPIN_LOCK`: Spin lock
  - `-DTTS_LOCK`: Test and test and set lock
  - `-DTICKET_LOCK`: Ticket lock
  - `-DARRAY_LOCK`: Array lock
  - `-DSEMAPHORE_LOCK`: Lock binary semaphore

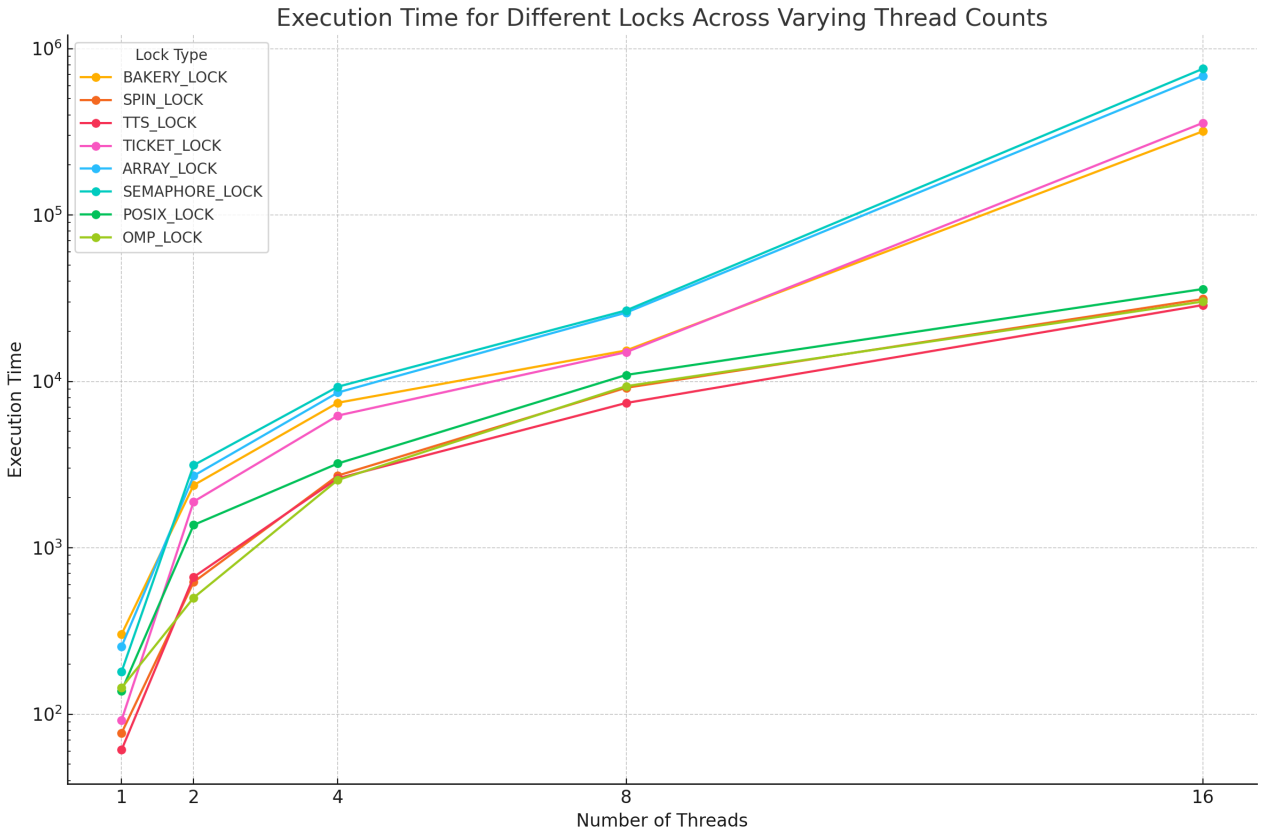
  For example, to test the Spin lock:
  `$ gcc -O3 -pthread -DSPIN_LOCK pthread_main_lock.c -o program`
  `$ ./program <num_threads>`
  The output will be as follows: `Time taken: <time> microseconds`
- `omp_main_lock.c`: Run the benchmark code using `#pragma omp critical`
- `pthread_main_barrier.c`: Similar to the locks, it has following flags to use different barrier implementations.
  - `-DCENTRALIZED_BW_BARRIER`: Centralized barrier using busy-wait on flags
  - `-DTREE_BW_BARRIER`: Tree barrier using busy wait on flags
  - `-DCENTRALIZED_CV_BARRIER`: Centralized barrier using condition variables
  - `-DTREE_CV_BARRIER`: Tree barrier using condition variables

  The usage is same as `pthread_main_lock.c`
- `omp_main_barrier.c`: Benchmark using `#pragma omp barrier`

2 Locks

2.1 Results

Lock Type	1 Thread	2 Threads	4 Threads	8 Threads	16 Threads
BAKERY_LOCK	302	2378	7438	15337	318116
SPIN_LOCK	77	622	2716	9151	31235
TTS_LOCK	61	667	2612	7414	28742
TICKET_LOCK	92	1894	6227	14981	357261
ARRAY_LOCK	255	2708	8569	25893	685214
SEMAPHORE_LOCK	180	3136	9284	26696	755812
POSIX_LOCK	138	1371	3209	10928	351886
OMP_LOCK	144	501	2555	9358	30166
BEST	TTS	OMP	TTS	TTS	TTS



2.2 Observations

- Test and test and set lock is working the best for almost any number of threads, although POSIX, openMP and Spin lock are quite close in terms of time.
  The reason is probably because of the very low overheads of cmpxchg instruction which leads to good efficiency of these locks and further TTS helps in avoiding certain cache invalidations which further boosts the performance.

- Also Ticket lock is quite slow because of an extra loop to simulate atomic fetch and increment instruction. Also even after a implementing an array lock to reduce the cache invalidations, still the time doesn't show an improvement.

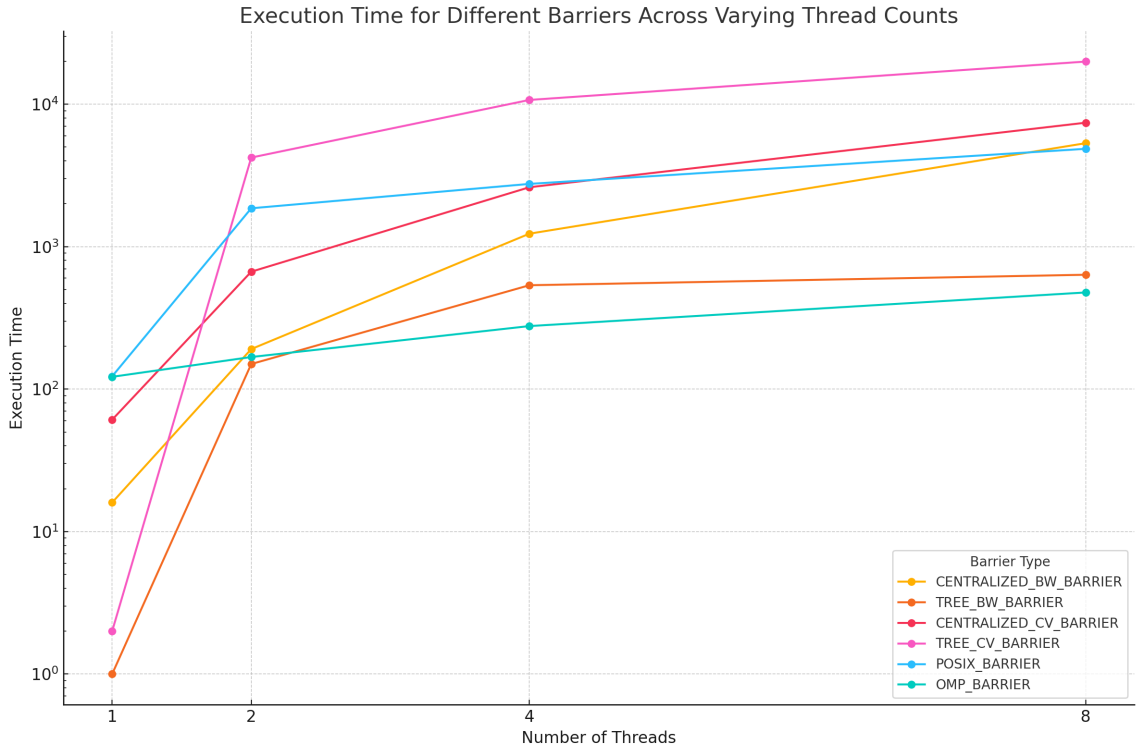
- Among all the locks, semaphore locks are the slowest which is also expected since they are more complex

- Bakery lock also does not perform very well because of complicated logic and need of multiple fence instructions as well. However, it is still better than Ticket and array locks and also semaphore based lock.
- Overall since the machine on which I tested had 4*2 cores. It was expected that beyond 8 threads, the time will see a sharper increase and we can also observe this from the empirical data.

3 Barriers

3.1 Results

Barrier Type	1 Thread	2 Threads	4 Threads	8 Threads
CENTRALIZED_BW_BARRIER	16	191	1233	5325
TREE_BW_BARRIER	1	150	536	635
CENTRALIZED_CV_BARRIER	61	667	2612	7414
TREE_CV_BARRIER	2	4220	10709	19912
POSIX_BARRIER	123	1859	2757	4862
OMP_BARRIER	122	168	277	477
BEST	TREE BW	TREE BW	OMP	OMP



3.2 Observations

- We can see that for multiple threads, openMP is working too good. this was also very surprising since I could not think of how openMP is managing to keep the time almost constant for any number of threads.
- The only implementation that was close to openMP is tree barrier with busy wait using flags. As we can see from the chart that it is almost matching openMP's time which is expected because of very low synchronization overheads. Since tree barrier with busy wait does not require any locks, most of the time is spent in waiting for all the threads to arrive at the barrier.
- Centralized barrier with busy wait also performed well in the start but it also has a synchronization overhead of acquiring a lock which makes it slower and comparable to the POSIX barrier in terms of time.

- Barriers with conditional variable are slower than busy wait barriers since all the threads will be reaching the barrier in almost the same time, synchronizing through conditional variables is a very big overhead and thus we can see the very high increment in running time.
- Also we can see that tree barrier with conditional variable is significantly slower as compared to centralized since here the bottleneck is the overhead of synchronization using cond_wait which is higher in a tree barrier as compared to centralized barrier due to the presence of multiple levels in trees.
- Lastly, I could not run the code for 16 threads since at that number of threads, half threads have to be context switched out. So at each barrier there have to be at least 8 context switches in order to cross the barrier. Thus totally 1e6 barriers would take a significantly much amount of time which I could not run the code for. I let the code run for almost 20 minutes but still even openMP which was performing very well till 8 threads could not complete execution.